

Arquitetura Técnica - Front-end Django + MQTT

Diagrama de Arquitetura Completa

...

CAMADA DE APRESENTAÇÃO

Browser (Cliente)

Dashboard HTML + Chart.js

Hero Panel Metrics Grid Temperature Chart

Botões de Comando

Frota Refrigerada (Fleet Cards)

FR-101 FR-204 FR-312

JavaScript (dashboard.js)

HTTP/HTTPS

CAMADA DE APLICAÇÃO

Django Application (Port 8000)

URLs & Routing (urls.py)

GET / GET /api/ POST /api/command/

dashboard() api_state() api_command()

Views (views.py)

dashboard(request):

api_state(request):

api_command(request):

State Management (state.py)

DashboardState (Dataclass)

set_connection(connected, message, source)

MQTT Client Management (mqtt.py)

ensure_client_started()

valida comando (ON|OFF|PING)

Subscribed Topics:

Published Topics:

paho-mqtt (TCP/IP)

CAMADA MQTT

MQTT Brokers (High Availability)

Primary Broker Secondary Broker (Fallback)

Topics: Topics Sincronizados

Clientes Conectados:

(pub/sub)

Topic Namespace: logistica/frio/

Produtor: Node-RED (Publisher)

Node-RED Sensor/ Relay/

(opcional) Publica Subscrive

Publica:

...

Fluxos de Dados Detalhados

...

MQTT Broker

Django MQTT Client (thread subscriber)

topic: "logistica/frio/temperatura"

ingest_message("logistica/frio/temperatura", "7.2")

STATE.lock.acquire() [thread-safe]

float("7.2") = 7.2

STATE.latest_status = "NORMAL"

STATE.latest_temperature = 7.2

_record_history_entry()

STATE.lock.release()

(4 segundos depois)

Browser fetch("/api/state/")

Django GET /api/state/

ensure_client_started() [idempotente]

get_snapshot()

JsonResponse com estado completo

JavaScript dashboard.js recebe JSON

renderState(payload.data)

document.getElementById("hero-temperature").textContent = "7.2 °C"

document.getElementById("metric-status").textContent = "NORMAL"

chart.data.datasets[0].data = [7.1, 7.15, 7.2, ...]

chart.update("none") [sem animação]

DOM atualizado

Usuário vê "Temperatura: 7.2°C, Status: NORMAL"

...

JavaScript button.addEventListener("click")

sendCommand("ON")

fetch("/api/command/", {

Django POST /api/command/

parse JSON: { "command": "ON" }

publish_command("ON")

normalize: "ON".upper() = "ON"

topic = "logistica/frio/comando"

payload = "ON"

Para cada broker:

client.connect(broker.host, broker.port)

client.publish(

set_last_command("ON")

client.disconnect()

return (True, "Comando ON enviado para logistica/frio/comando")

JsonResponse({"success": True, "message": "...", "command": "ON"})

pushEvent("Comando ON enviado para logistica/frio/comando", "event-status-ok")

MQTT Broker recebe publicação

Subscritos a "logistica/frio/comando":

Entrega mensagem [QoS=1]

Node-RED recebe mensagem

Node de input MQTT

Processa: payload = "ON"

Ativa função: ligar refrigeração

Relé/GPIO ativa

Refrigeração LIGADA

(opcional: enviar confirmação)

Node-RED publica em "logistica/frio/atuador/estado"

payload: "ATIVO" ou "ON"

Django subscreve e atualiza STATE.last_command

Arquitetura de Thread-Safety

...

Django Main Thread (Request Handling)

request handler

POST /api/command/

HTTP requests

MQTT Background Thread (Daemon)

_worker()

on_disconnect:

on_message:

client.loop_forever()

sleep(5) + retry

STATE (Compartilhado com main thread)

DashboardState

Acesso sincronizado:

Garantias de Thread-Safety:

Fluxo de Dados do Histórico

...

Recebimento 2: temp=7.1 @ 14:30:45

... (58 mais recebimentos) ...

Recebimento 60: temp=8.5 @ 15:10:30

get_snapshot() chamado:

Frontend recebe:

Resultado Visual:

Evolução da Temperatura

9.0

14:30 14:40 14:50 15:00 15:10

...

Segurança - Fluxo de Autenticação (Proposto)

...

Django verifica Authorization header

Não encontrado

JsonResponse({"success": false, "message": "Token necessário"}, status=401)

Usuário obtém token primeiro:

TokenObtainPairView

valida credenciais

retorna {"access": "eyJ...", "refresh": "eyJ..."}

Usuário envia comando com token:

Django verifica JWT

JWTAuthentication.authenticate()

IsAuthenticated.has_permission()

publish_command("ON")

sucesso

****Esta arquitetura garante**:**